

```

variable_name := structure_variable_type {value1,
value2...valuen}
type Books struct {
    title string
    author string
    subject string
    book_id int
}
func main() {
    var Book1 Books    /* Declare Book1 of type Book */
    var Book2 Books    /* Declare Book2 of type Book */

    /* book 1 specification */
    Book1.title = "Wings of Fire"
    Book1.author = " A.P. J. Abdul Kalam"
    Book1.subject = "An Autobiography"
    Book1.book_id = 63108786

    /* book 2 specification */
    Book2.title = "Academic English"
    Book2.author = "Pearson"
    Book2.subject = "Check your English Proficiency"
    Book2.book_id = 6495700

    /* print Book1 info */
    fmt.Printf( "Book 1 title : %s\n", Book1.title)
    fmt.Printf( "Book 1 author : %s\n", Book1.author)
    fmt.Printf( "Book 1 subject : %s\n", Book1.subject)
    fmt.Printf( "Book 1 book_id : %d\n", Book1.book_id)
}
o/p
Book 1 title : Wings of Fire
Book 1 author : A.P. J. Abdul Kalam
Book 1 subject : An Autobiography
Book 1 book_id : 63108786

```

Interfaces

Go programming provides another data type called interfaces, which are a set of method signatures. Interface specifies what methods a type should have, and the type decides how to implement these methods.

```

import "fmt"
/* define an interface */
type SalaryCalculator interface {
    CalculateSalary() int
}
/* define a permanent employee */
type Permanent struct {
    empId, basicpay, pf int
}

/* define a contract employee */
type Contract struct {

```

```

    empId, basicpay int
}
//salary of permanent employee is sum of basic pay and pf
func (p Permanent) CalculateSalary() int {
    return p.basicpay + p.pf
}

//salary of contract employee is the basic pay alone
func (c Contract) CalculateSalary() int {
    return c.basicpay
}

/* define a method for SalaryCalculator */
func getSal( salarycalculator SalaryCalculator) int {
    return salarycalculator.CalculateSalary()
}
func main() {
    perm := Permanent{empId:10, basicpay:40000, pf:8000}
    cont := Contract {empId:20, basicpay:25000}

    fmt.Printf("Permanent salary: %d\n", getSal(perm))
    fmt.Printf("Contract salary: %d\n", getSal(cont))
}

```

The Output will be:

- Permanent salary: 48000
- Contract salary: 25000

To reiterate, Go, which is sufficiently mature and stable has its own elegance and programming idioms that make the language productive and fun to code. Go also provides a comprehensive standard library with the core packages that programmers need to build real world and Web based programs. Though originally designed for systems programming, Go has become increasingly popular with Web application development, and can be found in many open source and commercial software applications. To sum up, Golang, with its concurrency, channels, interfaces, inheritance, unit testing and packages, is easier to learn for freshers, compared to experienced developers, because they can quickly understand and adapt. **END** 🐧

References

- [1] <https://gobyexample.com/>
- [2] <https://tour.golang.org/>
- [3] <https://www.tutorialspoint.com/go>
- [4] <https://golangbot.com/>
- [5] <https://play.golang.org/>
- [6] <https://blog.hackingcode.io/>

 By: Neethu C. Sekar

The author is an open source enthusiast, currently working as an assistant professor in the Department of Computer Science, AmalJyothi College of Engineering, Kerala. She can be reached at nitucskr@gmail.com.